UC San Diego

DSC 102
Systems for Scalable Analytics

Umesh Bellur
Alexander Kolovos

Practice Questions DSC 102 Spring 2026 - 2

Q1: Clock Speeds, Execution Time

A processor runs at **3 GHz**. A program has following instruction mix:

- 40% arithmetic instructions, **1 cycle each**
- 30% memory instructions, **5 cycles each**
- 30% control instructions, **2 cycles each**

Total instructions executed = $\mathbf{2 \times 10^9}$

Questions

1. What is the **average CPI (Clock cycles per instruction)?**
2. What is the **total number of cycles**?
3. What is the **execution time**?
4. If memory instructions increase to **50%**, with arithmetic dropping to **20%** and control staying at **30%**, compute new execution time.

Q1: Clock Speeds, Execution Time

A processor runs at **3 GHz**. A program has following instruction mix:

- 40% arithmetic instructions, **1 cycle each**
- 30% memory instructions, **5 cycles each**
- 30% control instructions, **2 cycles each**

Total instructions executed = **2×10^9**

Questions

1. What is the **average CPI (Clock cycles per instruction)?**
   CPI = (0.4)(1) + (0.3)(5) + (0.3)(2) = 2.5
2. What is the **total number of cycles**?
   Total Cycles = [Instruction Count] × CPI = $2\times10^9 \times 2.5 = 5\times10^9$ cycles
3. What is the **execution time**?
   Execution Time = [Total Cycles] / [Clock Rate] = $5\times10^9 / 3\times10^9$ = 5/3 or ~1.67 sec

Q1: Clock Speeds, Execution Time

A processor runs at **3 GHz**. A program has following instruction mix:

- 40% arithmetic instructions, **1 cycle each**
- 30% memory instructions, **5 cycles each**
- 30% control instructions, **2 cycles each**

Total instructions executed = **2×10^9**

Questions

4. If memory instructions increase to **50%**, with arithmetic dropping to **20%** and control staying at **30%**, compute new execution time.
   The new instruction mix has:
   Arithmetic: 20% at 1 cycle
   Memory: 50% at 5 cycles
   Control: 30% at 2 cycles
   New CPI: (0.2)(1) + (0.5)(5) + (0.3)(2) = 3.3
   New total cycles: 2×109 × 3.3 = 6.6×10^9 cycles
   New execution time: = 6.6×10^9 / 3×10^9 = 2.2 sec
   Increase in runtime: 2.2 − 1.67 = 0.53 sec
   Percentage increase: (2.2 – 1.67) / 1.67 = 0.31 or 31%

Q2: Memory Addressability

In a ARM-like system:

- Each instruction is **4 bytes**
- Instructions must be **4-byte aligned**

PC starts at **0x1000**

Questions

1. What are the next 5 instruction addresses?
2. How many instructions fit in **1 KB** of instruction memory?
3. What fraction of all 32-bit addresses are valid instruction addresses?

Q2: Memory Addressability

In a ARM-like system:

- Each instruction is **4 bytes**
- Instructions must be **4-byte aligned**

PC starts at **0x1000**

Questions

1. What are the next 5 instruction addresses?

   Each instruction is 4 bytes, so PC increases by 4 each time.

   Starting from 0x1000: 0x1000, 0x1004, 0x1008, 0x100C, 0x1010

2. How many instructions fit in **1 KB** of instruction memory?

   1 KB=1024 bytes. Each instruction is 4 bytes. Therefore 1024/4 = 256 instructions.

3. What percentage of all 32-bit addresses are valid instruction addresses?

   A 4-byte aligned address must end in binary 00, meaning the last 2 bits are fixed.

   Out of all possible 2-bit endings:

   00 is valid, 01 is invalid, 10 is invalid, and 11 is invalid.

   So only 1 out of 4 addresses are valid instruction addresses, or 25%.

Q3: Memory Dominates Performance

A program executes:

- **1 billion instructions**
- 30% are memory operations
- Each memory operation stalls for **100 cycles**
- CPU clock = **2.5 GHz**
- Non-memory instructions take **1 cycle**
- Assume memory instructions also take **1 base cycle** plus the stall

Questions

1. How many cycles are spent on base instruction execution?
2. How many cycles are spent waiting on memory?
3. What is the total execution time?
4. What percentage of total cycles are memory stall cycles?
5. If cache reduces stall from 100 cycles to 20 cycles, compute speedup.

Q3: Memory Dominates Performance

A program executes:

- **1 billion instructions**
- 30% are memory operations
- Each memory operation stalls for **100 cycles**
- CPU clock = **2.5 GHz**
- Non-memory instructions take **1 cycle**
- Assume memory instructions also take **1 base cycle** plus the stall

Questions

Observe that: Total instructions: 10^9.

Of those: Memory: $0.3\times10^9 = 3\times10^8$, so non-memory: 7×10^8

1. How many cycles are spent on base instruction execution?
   All instructions take at least 1 cycle. Therefore: 10^9 instructions * 1 = 10^9 cycles
2. How many cycles are spent waiting on memory?
   Each memory instruction adds 100 stall cycles: 3×10^8 * 100 = 3×10^{10} cycles
3. What is the total execution time?
   Total cycles: $10^9 + 3\times10^{10} = 3.1\times10^{10}$. Also, clock rate: 2.5×10^9 cycles per second
   Execution time = [Total cycles] / [Clock rate] = $3.1\times10^{10} / 2.5\times10^9$ = 12.4 seconds

Q3: Memory Dominates Performance

A program executes:

- **1 billion instructions**
- 30% are memory operations
- Each memory operation stalls for **100 cycles**
- CPU clock = **2.5 GHz**
- Non-memory instructions take **1 cycle**
- Assume memory instructions also take **1 base cycle** plus the stall

Questions

1. How many cycles are spent on base instruction execution? 10^9 cycles
2. How many cycles are spent waiting on memory? 3×10^{10} cycles
3. What is the total execution time? 12.4 seconds
4. What percentage of total cycles are memory stall cycles?
   From the responses in [1] and [2], total number of cycles is $10^9 + 3\times10^{10} = 3.1\times10^{10}$
   Therefore, $3\times10^{10} / 3.1\times10^{10} = 3/3.1 =$ 0.97 or 97%
5. If cache reduces stall from 100 cycles to 20 cycles, compute speedup.
   The stall cycles will be $3\times10^8 * 20 = 6\times10^9$ for a total of $10^9 + 6\times10^9 = 7\times10^9$ cycles and exec time $7\times10^9 / 2.5\times10^9 = 2.8$ sec. Speedup is 12.4 / 2.8 = 4.43x faster.

Q4: Caching And Avg Access Time

1. Assume a two-level hierarchy: CPU cache and Memory (DRAM) only. Given
   CPU cache access time: 10 ns with hit rate: 0.7
   Memory access time: 1.5 μs
   What is the average access time?

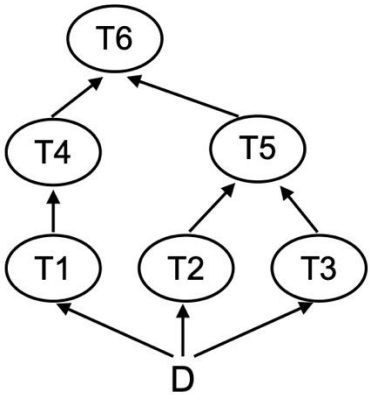
2. Assume a three-level hierarchy: CPU cache L1, L2, and Memory (DRAM). Given
   CPU L1 cache access time: 10 ns with hit rate: 0.7
   CPU L2 cache access time: 70 ns with hit rate: 0.9
   Memory access time: 1.5 μs
   What is the average access time?

Q4: Caching And Avg Access Time

1. Assume a two-level hierarchy: CPU cache and Memory (DRAM) only. Given
   CPU cache access time: 10 ns with hit rate: 0.7
   Memory access time: 1.5 μs
   What is the average access time?
   The average access time is the sum of weighted access times for L1, DRAM.
   Using ns as a time unit, we have 1.5μs=1500ns, and therefore:
   0.7*10 + (1-0.7)*1500 = 0.7 + 0.3*1500 = 0.7 + 450 = 457 ns

2. Assume a three-level hierarchy: CPU cache L1, L2, and Memory (DRAM). Given
   CPU L1 cache access time: 10 ns with hit rate: 0.7
   CPU L2 cache access time: 70 ns with hit rate: 0.9
   Memory access time: 1.5 μs
   What is the average access time?
   The average access time is the sum of weighted access times for L1, L2, DRAM.
   Using ns as a time unit, we have 1.5μs=1500ns, and therefore:
   0.7*10 + (1-0.7)*[0.9*70 + (1-0.9)*1500] =
   = 0.7 + 0.3*(63 + 0.1*1500) =
   = 0.7 + 0.3*(63 + 150) = 0.7 + 0.3*213 = 0.7 + 63.9 = 64.6 ns

Q5: Task Graphs - 1



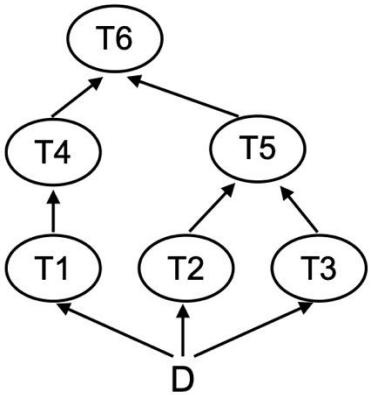
Task	Length
T1	10
T2	15
T3	10
T4	20
T5	10
T6	5

In following exercises, assume task parallelism as discussed in class (i.e., no preemption or migration of tasks).

Consider the task graph shown on the left, the given task lengths in the table (in time units), and answer the following questions.

1. What is the largest degree of parallelism, if you were to execute this workload using task parallelism?
2. Suppose you are given 3 identical worker nodes and use task parallelism. What is the lowest possible completion time of this workload?
3. Continuing with the above question's setup, what is the highest possible speedup against running on only one worker node?
4. Continuing with the above question's setup, in a task-parallel schedule that offers the highest possible speedup, what is the highest possible idle time of one worker among the 3 workers? Assume all workers on, from start to the end of the whole workload.
5. Now suppose you are given only 2 identical worker nodes for task parallelism. What is the lowest possible completion time of this workload now?

Q5: Task Graphs - 1



Task	Length
T1	10
T2	15
T3	10
T4	20
T5	10
T6	5

1. What is the largest degree of parallelism, if you were to execute this workload using task parallelism?

Answer: 3 (Because all tasks T1, T2, and T3 can run in parallel.)

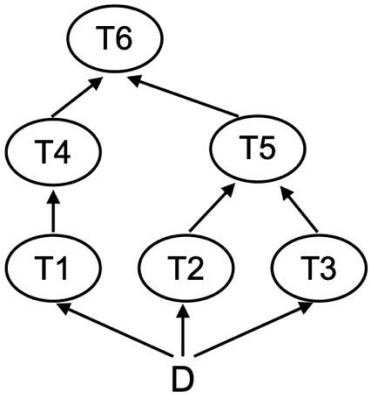
2. Suppose you are given 3 identical worker nodes and use task parallelism. What is the lowest possible completion time of this workload?

Answer: 35 (We need to find the longest path from D to end, which is T1 →T4 →T6.)

3. Continuing with the above question's setup, what is the highest possible speedup against running on only one worker node?

Answer: 2x (Total time of all tasks on a single node is 70; thus, speedup is 70/35 = 2x.)

Q5: Task Graphs - 1



Task	Length
T1	10
T2	15
T3	10
T4	20
T5	10
T6	5

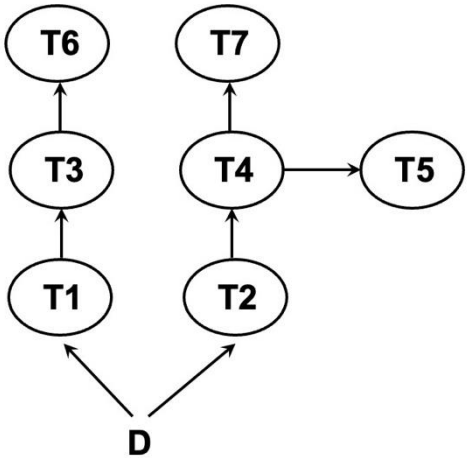
4. Continuing with the above question's setup, in a task-parallel schedule that offers the highest possible speedup, what is the highest possible idle time of one worker among the 3? Assume all workers on, from start to the end of the whole workload.

Answer: 25 (T1, T4, T6 on W1; T2 and T5 on W2; T3 on W3. Idle time is largest on W3: 35−10= 25.)

5. Now suppose you are given only 2 identical worker nodes for task parallelism. What is the lowest possible completion time of this workload now?

Answer: 40 (With only 2 workers, one best possible schedule places T3 after T2 and before T5 on W2, while W1 runs T1 and T4. Note that T6 cannot start until both T4 and T5 finish. So, new overall lowest completion time possible is 35 (T2, T3, T5 on W2) + 5 (for T6 on any of the workers).)

Q6: Task Graphs - 2



Task	Length
T1	15
T2	5
T3	10
T4	x
T5	10
T6	10
T7	10

Consider the above task graph and given task lengths (in time units). The value of x is a non-negative number that can vary as specified in each of the following questions.

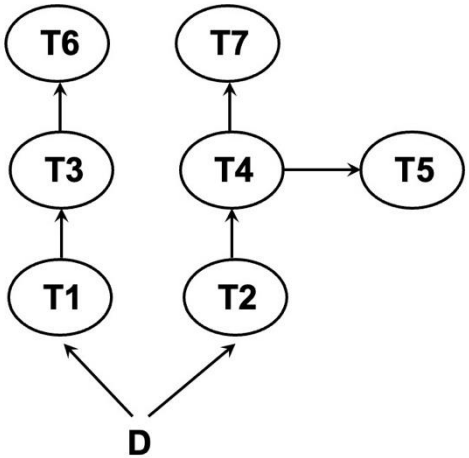
1. What is the degree of parallelism of this workload for task-parallel execution?

Answer: 3 (The figure shows that tasks T5, T6, and T7 can be executed independently).

2. Assume x=20. Suppose the workload is executed in a task-parallel manner for the lowest possible completion time on 3 workers. All workers are on until the last task finishes. What is the total idle time (add across workers)?

Answer: 25 (If we have 3 workers: T1-T3-T6 and T2-T4-T7 can run in parallel on 2 workers, followed by T5 running on the 3rd worker. T5 needs to wait for 25 units to begin, and so do T6 and T7. T5, T6, and T7 finish at the same time. So, total idle time is the wait time for T1-T3 or T2-T4, which is 25.)

Q6: Task Graphs - 2



Task	Length
T1	15
T2	5
T3	10
T4	x
T5	10
T6	10
T7	10

3. Assume x=20. What is the lowest possible completion time of this workload with task-parallelism?

Answer: 35 (We need to find the longest single path for a worker. In the present scenario, it will take for all workers 35 units to complete their tasks.)

4. Assume x=5. What is the value of the lowest possible completion time with task-parallelism when given only 2 workers?

Answer: 35 (If we only have 2 workers, then the path T1-T3-T6 takes 35 units for one worker; the path T2-T4-T7 (or T2-T4-T5) takes 20 units for the second worker. The second worker will finish earlier and can do the remaining task T5 (or T7) within 10 more units, thus totaling 30 units. As a result, the lowest possible completion given 2 workers will take 35 units, because this is the longest path needed to be followed (the one done by the first worker).